dYdX v4 Protocol Layer - Cosmos SDK 기반 모듈 구조 분석

개요

dYdX v4 Protocol은 Cosmos SDK와 CometBFT를 기반으로 구축된 주권형 블록체인으로, 고성능 탈중앙화 영구 선물 거래소를 제공합니다. 이 문서는 Protocol Layer의 Cosmos SDK 기반 모듈 구조를 상세히 분석하여, 각 모듈의 역할과 기능, 모듈 간 의존성 및 상호작용 관계, 그리고 Genesis 설정 및 초기화 과정을 설명합니다.

전체 모듈 구조

dYdX v4 Protocol은 표준 Cosmos SDK 모듈과 dYdX 특화 커스텀 모듈로 구성됩니다.

표준 Cosmos SDK 모듈

```
protocol/
├── x/auth # 계정 인증 및 관리
├── x/authz # 권한 위임
├── x/bank # 토큰 전송 및 잔고 관리
├── x/staking # 검증자 스테이킹
├── x/slashing # 검증자 처벌
├── x/distribution # 보상 분배
├── x/gov # 거버넌스
├── x/crisis # 위기 상황 처리
```

```
├── x/upgrade # 체인 업그레이드
├── x/evidence # 증거 제출
├── x/feegrant # 수수료 지원
├── x/params # 매개변수 관리
├── x/consensus # 합의 매개변수
├── x/capability # 기능 관리
└── IBC 모듈들 # 크로스체인 통신
```

dYdX 커스텀 모듈

```
protocol/x/
├── assets/ # 자산 관리
├── blocktime/ # 블록 시간 관리
├── bridge/ # 크로스체인 브릿지
├── clob/ # 중앙 지정가 주문서 (CLOB)
├── delaymsg/ # 지연 메시지 처리
├── epochs/ # 에포크 관리
├── feetiers/ # 수수료 등급
├── govplus/ # 확장된 거버넌스
├── perpetuals/ # 영구 선물 계약
├── prices/ # 가격 피드
├── ratelimit/ # 속도 제한
├── rewards/ # 보상 시스템
```

```
├── sending/ # 전송 관리
├── stats/ # 통계 수집
├── subaccounts/ # 서브계정 관리
├── vault/ # 볼트 관리
└── vest/ # 베스팅 관리
```

핵심 dYdX 모듈 상세 분석

1. CLOB 모듈 (Central Limit Order Book)

위치: `protocol/x/clob/`

역할: 중앙 지정가 주문서 관리 및 매칭 엔진 구현

주요 기능:

- 주문 생성, 수정, 취소 처리
- 가격-시간 우선순위 기반 매칭 알고리즘
- 메모리 기반 주문서 (MemCLOB) 관리
- 조건부 주문 처리

생명주기 메서드:

- `BeginBlocker` : 블록 시작 시 초기화 작업
- `EndBlocker` : 블록 종료 시 만료된 주문 정리, 조건부 주문 트리거
- `PrepareCheckState` : 다음 블록을 위한 상태 준비

의존성:

- `subaccounts` : 계정 잔고 및 포지션 확인

- `assets` : 자산 정보 조회
- `perpetuals` : 영구 선물 계약 정보
- `prices` : 시장 가격 정보
- `feetiers` : 수수료 계산

2. Prices 모듈

위치: `protocol/x/prices/`

역할: 외부 거래소 가격 데이터 관리 및 오라클 기능

주요 기능:

- 다중 거래소 가격 집계
- 가격 검증 및 이상치 탐지
- 스무딩된 가격 계산
- 오라클 가격 업데이트

데이터 구조:

- `MarketPrice` : 시장별 가격 정보
- `MarketParam` : 시장 매개변수
- `SmoothedPrices` : 스무딩된 가격 데이터

의존성:

- Price Daemon과 gRPC 통신
- Indexer Event Manager와 연동

3. Perpetuals 모듈

위치: `protocol/x/perpetuals/`

역할: 영구 선물 계약 관리

주요 기능:

- 펀딩 레이트 계산
- 포지션 관리
- 마진 요구사항 계산
- 청산 조건 모니터링

생명주기 메서드:

- `EndBlocker` : 펀딩 레이트 업데이트

의존성:

- `prices` : 시장 가격 정보
- `epochs` : 펀딩 레이트 업데이트 주기
- `clob` : 매칭 엔진과 연동

4. Subaccounts 모듈

위치: `protocol/x/subaccounts/`

역할: 사용자 계정 및 잔고 관리

주요 기능:

- 다중 자산 잔고 관리
- 포지션 추적
- 마진 계산
- 계정 업데이트 이벤트 생성

데이터 구조:

- `Subaccount` : 서브계정 정보
- `AssetPosition` : 자산 포지션

- `PerpetualPosition` : 영구 선물 포지션

의존성:

- `assets` : 자산 정보
- `perpetuals` : 영구 선물 정보
- `bank` : 토큰 전송

5. Assets 모듈

위치: `protocol/x/assets/`

역할: 거래 가능한 자산 관리

주요 기능:

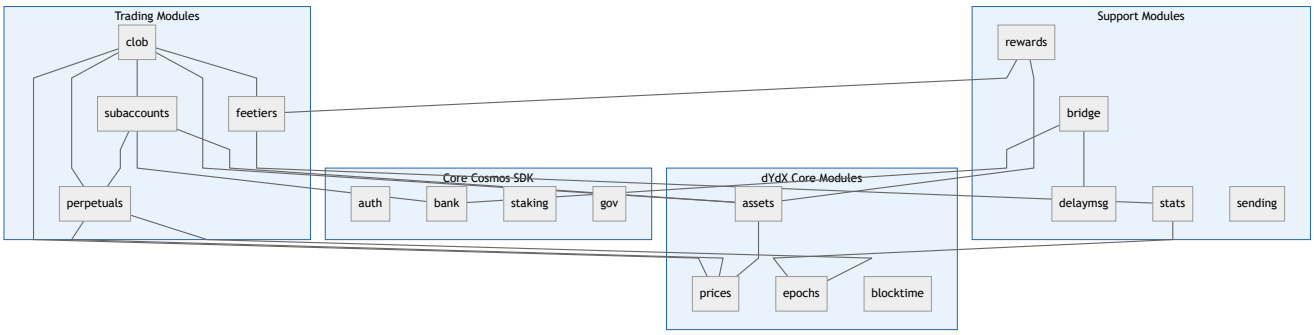
- 자산 등록 및 관리
- 자산 메타데이터 저장
- 자산별 설정 관리

의존성:

- `prices` : 자산 가격 정보

모듈 간 의존성 관계

의존성 그래프



주요 의존성 설명

1. **CLOB → 다른 모듈들**: CLOB은 거래 처리를 위해 거의 모든 다른 모듈에 의존
2. **Subaccounts → Assets, Perpetuals**: 계정 관리를 위해 자산과 포지션 정보 필요
3. **Perpetuals → Prices, Epochs**: 펀딩 레이트 계산을 위해 가격과 시간 정보 필요
4. **Assets → Prices**: 자산 가치 평가를 위해 가격 정보 필요

Genesis 설정 및 초기화 과정

Genesis 초기화 순서

`app.go` 에서 정의된 모듈 초기화 순서:

```
app.ModuleManager.SetOrderInitGenesis(

epochsmoduletypes.ModuleName, // 1. 에포크 (시간 기준)
```

```
capabilitytypes.ModuleName, // 2. 기능 관리
authtypes.ModuleName, // 3. 계정 인증
banktypes.ModuleName, // 4. 토큰 관리
distrtypes.ModuleName, // 5. 보상 분배
stakingtypes.ModuleName, // 6. 스테이킹
slashingtypes.ModuleName, // 7. 슬래싱
govtypes.ModuleName, // 8. 거버넌스
// ... IBC 모듈들
pricesmoduletypes.ModuleName, // 가격 피드
assetsmoduletypes.ModuleName, // 자산 관리
blocktimemoduletypes.ModuleName, // 블록 시간
bridgemoduletypes.ModuleName, // 브릿지
feetiersmoduletypes.ModuleName, // 수수료 등급
perpetualsmoduletypes.ModuleName, // 영구 선물
statsmoduletypes.ModuleName, // 통계
satypes.ModuleName, // 서브계정
clobmoduletypes.ModuleName, // CLOB (마지막)
// ... 기타 모듈들
)
```

초기화 순서의 중요성

1. **기본 모듈 우선**: `epochs`, `auth`, `bank` 등 기본 기능 모듈이 먼저 초기화

2. **의존성 순서**: 의존하는 모듈이 먼저 초기화 (예: `assets` → `subaccounts` → `clob`)

3. **CLOB 마지막**: 가장 복잡하고 많은 의존성을 가진 CLOB 모듈이 마지막에 초기화

Genesis 상태 구조

각 모듈은 고유한 Genesis 상태를 가지며, 전체 Genesis는 다음과 같이 구성됩니다:

```
{
"app_name": "dydxprotocol",
"app_version": "...",
"genesis_time": "...",
"chain_id": "...",
"app_state": {
"auth": { /* 계정 정보 */ },
"bank": { /* 토큰 공급량 */ },
"assets": { /* 자산 목록 */ },
"prices": { /* 초기 가격 */ },
"perpetuals": { /* 영구 선물 계약 */ },
"clob": { /* 주문서 설정 */ },
"subaccounts": { /* 계정 상태 */ }
// ... 기타 모듈 상태
}
}
```

블록 생명주기에서의 모듈 실행 순서

BeginBlocker 순서

```
app.ModuleManager.SetOrderBeginBlockers(
blocktimemoduletypes.ModuleName, // 블록 시간 업데이트 (최우선)
authz.ModuleName, // 만료된 권한 삭제
epochsmoduletypes.ModuleName, // 에포크 업데이트
// ... 표준 Cosmos SDK 모듈들
pricesmoduletypes.ModuleName, // 가격 업데이트
assetsmoduletypes.ModuleName, // 자산 상태 업데이트
// ... 기타 dYdX 모듈들
)
```

EndBlocker 순서

```
app.ModuleManager.SetOrderEndBlockers(
// ... 표준 모듈들
clobmoduletypes.ModuleName, // CLOB 정리 작업
perpetualsmoduletypes.ModuleName, // 펀딩 레이트 업데이트
// ... 기타 모듈들
blocktimemoduletypes.ModuleName, // 블록 시간 마무리 (최후)
)
```

PrepareCheckState

CLOB 모듈만이 `PrepareCheckState` 를 구현하여 다음 블록을 위한 메모리 상태를 준비합니다.

모듈별 Store Key 관리

각 모듈은 고유한 Store Key를 가지며, 데이터 격리를 보장합니다:

```
keys := storetypes.NewKVStoreKeys(
// 표준 모듈들
authtypes.StoreKey,
banktypes.StoreKey,
// dYdX 모듈들
pricesmoduletypes.StoreKey,
assetsmoduletypes.StoreKey,
perpetualsmoduletypes.StoreKey,
satypes.StoreKey, // subaccounts
clobmoduletypes.StoreKey,
// ... 기타
)
```

Transient Store Keys

일부 모듈은 블록 내에서만 유지되는 임시 저장소를 사용합니다:

```
tkeys := storetypes.NewTransientStoreKeys(
```

```
clobmoduletypes.TransientStoreKey, // CLOB 임시 데이터

statsmoduletypes.TransientStoreKey, // 통계 임시 데이터

rewardsmoduletypes.TransientStoreKey, // 보상 임시 데이터

)
```

Memory Store Keys

메모리 기반 저장소는 주로 CLOB 모듈에서 사용됩니다:

```
memKeys := storetypes.NewMemoryStoreKeys(

clobmoduletypes.MemStoreKey, // MemCLOB 데이터

)
```

결론

dYdX v4 Protocol의 모듈 구조는 Cosmos SDK의 모듈화 아키텍처를 활용하여 각 기능을 독립적이면서도 상호 연동되는 모듈로 구성했습니다. 특히 CLOB 모듈을 중심으로 한 거래 관련 모듈들의 의존성 관계와 초기화 순서가 체계적으로 설계되어 있어, 고성능 탈중앙화 거래소의 요구사항을 충족할 수 있도록 구현되었습니다.